

Semana 8

Desarrollo Backend III

(PBY2203)

Matías Zúñiga

11/10/2026

Modernización Backend Banco XYZ

1. Resumen ejecutivo

El proyecto moderniza procesos bancarios legacy mediante Spring Batch y una arquitectura distribuida basada en microservicios. La solución incorpora BFF independientes para Web, Mobile y ATM, seguridad OAuth2, configuración centralizada, descubrimiento de servicios, resiliencia, mensajería Kafka, PostgreSQL y contenedores Docker.

2. Procesos críticos y arquitectura

Se abordaron cinco procesos críticos de modernización:

1. Migración de procesos legacy a Spring Batch.
2. División de la lógica bancaria en microservicios independientes.
3. Implementación de BFF para Web, Mobile y ATM.
4. Seguridad distribuida mediante Spring Security y OAuth2.
5. Mensajería asíncrona mediante Apache Kafka.

La arquitectura responde a tres requisitos principales:

Requisito	Decisión
Mantener integridad y capacidad de reproceso	Spring Batch con metadata persistente, manejo de errores e idempotencia
Separar canales y responsabilidades de negocio	BFF independientes y Account, Payment y Customer Service
Mantener disponibilidad, seguridad y escalabilidad	OAuth2, Eureka, Config Server, Resilience4j, Kafka y Docker

La solución utiliza PostgreSQL compartido como etapa de modernización y separa la lógica de negocio en account-service, payment-service y customer-service.

3. Migración Spring Batch

Se implementaron tres jobs sobre los datos legacy oficiales.

3.1 Reporte diario de transacciones

Procesa movimientos_financieros_diarios.csv mediante Reader, Processor y Writer. De 1.000 registros se procesaron 392 y se descartaron 608 registros inválidos controladamente. El resultado quedó persistido para 239 días.

3.2 Cálculo de intereses

Procesa intereses_trimestrales.csv. De 1.000 registros se escribieron 296 y se descartaron 704 registros inválidos. El resultado consolidó 50 cuentas únicas sin diferencias de integridad.

3.3 Estados de cuenta anuales

Procesa estados_financieros_anuales.csv. De 1.000 registros se escribieron 836 y se descartaron 164. El resultado final contiene 819 cuentas únicas sin inconsistencias detectadas.

3.4 Tolerancia a fallos y escalabilidad

Los tres jobs utilizan procesamiento por chunks, políticas de skip y retry, listeners de ejecución y metadata persistente de Spring Batch. Los archivos se dividen mediante Partitioner y se procesan con un TaskExecutor, permitiendo ejecución paralela.

La persistencia utiliza operaciones idempotentes para permitir reejecución sin duplicar resultados. JobRepository conserva el estado de ejecución y permite controlar finalización y reinicio de los jobs.

4. BFF y seguridad

Web, Mobile y ATM poseen BFF independientes y respuestas adaptadas a cada canal. Los accesos se protegen mediante OAuth2/JWT con scopes web, mobile y atm, y los tokens se propagan hacia los microservicios protegidos.

5. Microservicios, resiliencia y mensajería

account-service gestiona cuentas y saldos, payment-service procesa operaciones monetarias y customer-service administra información de clientes.

Spring Cloud Config centraliza configuración y Eureka permite descubrimiento y balanceo entre instancias. Resilience4j aplica Circuit Breaker y Bulkhead; Retry se utiliza sólo donde la operación es segura de repetir.

payment-service publica eventos en Kafka mediante bank.withdrawals y bank.payments. Los retiros pueden ser procesados asíncronamente por múltiples instancias de withdrawal-consumer.

6. Contenedores y escalabilidad

Los componentes se ejecutan mediante Docker Compose. Los tres microservicios de negocio poseen imágenes multi-stage, health checks y ejecución no root.

Account, Payment y Customer Service fueron validados con dos instancias simultáneas de cada servicio registradas en Eureka, comprobando escalabilidad horizontal y descubrimiento independiente.

7. Resultados y mejoras futuras

La solución permite ejecutar los procesos Batch, acceder a los servicios mediante BFF protegidos, tolerar fallos de dependencias y procesar eventos financieros de forma asíncrona.

Como evolución productiva se propone persistir los clientes del Authorization Server, incorporar Authorization Code con PKCE para usuarios finales, aplicar un patrón Outbox

para coordinar persistencia y eventos, evolucionar hacia persistencia independiente por microservicio e incorporar métricas y trazabilidad distribuida.